Aalto-yliopisto
Aalto-universitetet
Aalto University

Bachelor's Programme in Science and Technology

Context Engineering for AI-Assisted Software Development

Juuso Mäkinen

Bachelor's thesis
2025

Aalto-yliopisto
Aalto-universitetet
Aalto University

Author Juuso Mäkinen

Title of thesis Context Engineering for AI-Assisted Software Development

Programme Bachelor's Programme in Science and Technology

Major Computer Science

Thesis supervisor Professor Lauri Savioja

Thesis advisors Master of Science Yu Cong and Doctor of Science Pekka Alho

Collaborative partner ABB

Date 14.12.2025 **Number of pages** 41 **Language** English

Abstract

Currently the most advanced large language models are based on neural networks, the self-attention mechanism, and transformer architecture. However, the self-attention mechanism causes a significant limitation for these language models: computational and memory requirements grow quadratically as the input given to the language model grows. This quadratic growth causes significant scalability problems with development of more powerful models. Another notable problem is the weakening of performance with long inputs; a phenomenon termed Lost-in-the-middle. Studies show that the ability of the model to manage details weakens considerably when the relevant information is located in the middle part of the context window.

Context engineering is a modern concept that covers all methods that aim to enhance the ability of large language models to manage their context. Context refers to all possible information that an AI model can utilize in the generation task given to it. Such information can be, for example, in the context window of the model, in memory, in the model's parameters, or in an external database.

This bachelor's thesis is a literature review of the most commonly used context engineering sub-areas and their best implementations. The source material combines theoretically strong basic research and pragmatic, topical solution proposals that have not yet been extensively addressed in established academic literature. Due to the rapid pace of development in this field, this review synthesizes peer-reviewed literature with recent pre-prints and technical reports to capture state-of-the-art methodologies. This thesis analyzes both widely accepted methods and unique newer solutions.

This thesis aims to identify deployable context engineering methods that improve the performance of large language models, especially in tasks related to software development with large code repositories. In this thesis, the most important criteria for a good method are the adoptability of the method as well as the magnitude of the performance improvement it provides to the language model.

Based on these criteria, the analysis identifies that the best context engineering methods are prompt engineering and retrieval-augmented generation. Prompt engineering is an agile and adoptable method that offers a significant enhancement to the performance of the model. Retrieval-augmented generation is a method that augments the performance of the model by retrieving information for the model dynamically, for example by querying a database. This information given to the AI model can improve the performance in situations where it must process previously unknown, rare or proprietary information.

Keywords context engineering, prompt engineering, retrieval-augmented generation, context window

Aalto-yliopisto
Aalto-universitetet
Aalto University

Tekijä	Juuso Mäkinen
Työn nimi	Context engineering AI-avusteisessa ohjelmistokehityksessä
Koulutusohjelma	Teknillistieteellinen kandidaattiohjelma
Pääaine	Tietotekniikka
Vastuuopettaja/valvoja	Professori Lauri Savioja
Työn ohjaajat	Diplomi-insinööri Yu Cong ja Tekniikan tohtori Pekka Alho
Yhteistyötaho	ABB
Päivämäärä 14.12.2025	**Sivumäärä** 41 **Kieli** Englanti

Tiivistelmä

Nykyisin tehokkaimmat suuret kielimallit pohjautuvat neuroverkkoihin, itsehuomiomekanismiin ja transformer-arkkitehtuuriin. Etenkin itsehuomiomekanismi aiheuttaa näille kielimalleille merkittävän rajoitteen: laskentatehon ja muistin tarpeet kasvavat neliöllisesti kielimallille annetun syötteen kasvaessa. Tästä neliöllisestä kasvusta aiheutuu merkittäviä skaalautuvuusongelmia tekoälymallien kehityksessä. Toinen merkittävä ongelma on suorituskyvyn heikkeneminen pitkillä syötteillä, ilmiö joka tunnetaan nimellä ”Lost-in-the-middle”. Tutkimukset osoittavat, että mallin kyky hallinnoida yksityiskohtia heikkenee huomattavasti, kun relevantti tieto sijaitsee konteksti-ikkunan keskiosassa.

Context engineering on moderni termi, joka kattaa kaikki menetelmät, joilla pyritään parantamaan suurten kielimallien kykyä hallinnoida sen kontekstia. Kontekstilla tarkoitetaan kaikkea mahdollista tietoa, jota tekoälymalli voi hyödyntää sille annetussa generointitehtävässä. Tällaista tietoa voi olla esimerkiksi kielimallin konteksti-ikkunassa, muistissa, mallin parametreissa tai ulkoisessa tietokannassa.

Tämä kandidaatintyö on kirjallisuuskatsaus yleisimmin käytetyistä context engineering -osa-alueista ja niiden parhaista implementaatioista. Lähdeaineisto yhdistelee teoreettisesti vahvaa perustutkimusta ja pragmaattisia, ajankohtaisia ratkaisuehdotuksia, joita ei ole vielä käsitelty laajasti akateemisessa valtavirrassa. Monet moderneimmat ratkaisut juuri tämän aiheen parissa ovat hiljattain julkaistuja, eikä niitä välttämättä ole ehditty vertaisarvioida. Tässä työssä hyödynnetään monipuolisesti sekä laajalti hyväksyttyjä metodeja, mutta myös uniikkeja ja uudempia ratkaisuja.

Tässä selvityksessä pyritään löytämään ratkaisuja käyttöönotettavista context engineering -menetelmistä, jotka parantavat suurten kielimallien suorituskykyä erityisesti ohjelmistokehitykseen liittyvissä tehtävissä suurten koodivarastojen yhteydessä. Tässä työssä tärkeimpinä kriteereinä hyvälle menetelmälle ovat menetelmän käyttöönotettavuus sekä sen kielimallille antama suorituskykyparannuksen suuruus.

Näillä kriteereillä parhaiksi menetelmiksi osoittautuivat kehotesuunnittelu (eng. prompt engineering) sekä retrieval-augmented generation. Kehotesuunnittelu osoittautui erittäin ketterästi hyödynnettäväksi menetelmäksi, jolla kielimallien suorituskykyä voi huomattavasti parantaa. Retrieval-augmented generation on menetelmä, jolla tekoälymallille voi antaa tietoa dynaamisesti esimerkiksi tekemällä hakuja tietokannasta. Tämä tekoälymallille annettu tieto voi parantaa suorituskykyä muun muassa tilanteissa, joissa sen tulee käsitellä entuudestaan tuntematonta, harvinaista tai yksityistä tietoa.

Avainsanat context engineering, prompt engineering, retrieval-augmented generation, konteksti-ikkuna

Table of contents

Preface and acknowledgements

I want to thank the course staff and my thesis advisors Yu Cong and Pekka Alho for their good advice and guidance.

I also want to thank my partner for their support.

Otaniemi, 14 December 2025
Juuso Mäkinen

Abbreviations

AI	Artificial Intelligence
BM25	Best Matching 25
CAG	Cache-Augmented Generation
CAMELoT	Consolidated Associative Memory Enhanced Long Transformer
CCG	Code Context Graph
CE	Context Engineering
DAIL	Demonstration Augmentation for In-context Learning
FIFO	First-In-First-Out
GPT	Generative Pre-trained Transformer
ICL	In-Context Learning
LAIL	LLM-Aware In-Context Learning
LLM	Large Language Model
RACG	Retrieval-Augmented Code Generation
RAG	Retrieval-Augmented Generation
RLPG	Repo-Level Prompt Generator

1 Introduction

1.1 Background and Motivation

Context engineering is an emerging discipline on effectively leveraging large language models (LLMs) for code generation, particularly in environments with large and complex software repositories. Modern transformer-based LLM agents, such as GitHub Copilot [1] and Codex [2], can generate high-quality code when provided with appropriate context. However, the size and complexity of many industrial codebases pose unique challenges for *artificial intelligence-assisted* (AI-assisted) code generation. Such problems include repository-specific conventions, large modular structures with strict compatibility across the modules, dependencies, limitations and large multi-lingual repositories.

Several strategies have emerged to address context engineering challenges with LLMs. Mei et al. [3] introduce a taxonomy for context engineering that divides it into three foundational components: context generation and retrieval, context processing and context management. For example, the widely used prompt engineering techniques are a part of the context generation and retrieval component. The field of context engineering is fragmented. However, this thesis utilizes the taxonomy to create structure around the concepts.

In-context learning (ICL) is the capability of modern LLMs to learn during inference [4]. It enables developers to encode preferred behaviors, repository-specific guidelines and style rules directly into the prompt of the model or into a pre-prompt file without the need for traditional training of the model. In-context learning is lightweight, user-friendly and provides adaptable improvement. Furthermore, ICL is the platform upon which many prompt engineering and augmentation methodologies depend on. Therefore, ICL is vital to guide AI driven software development.

In addition to retrieval-based methods and prompt engineering, fine-tuning allows models to adapt to specific contexts, such as a proprietary codebase of an enterprise, through deep learning of the domain specifics during extended training with enterprise-specific data. Fine-tuning requires a pre-trained LLM, such as GPT-5 [5], which is then trained on the context-specific training set.

Despite rapid advancements in the field of AI, there is no standardized way for improving and implementing context engineering in industrial software development environments. Developers and organizations often rely on

heuristics [6], repetition and manual context selection, which can lead to inconsistent outputs and unreliable code production. Understanding the strengths, limitations and trade-offs between different context engineering techniques is essential for maximizing the reliability and productivity of AI-assisted software development.

1.2 Objectives and Research Questions

This thesis conducts a literature review of context engineering techniques for software development and focuses on large industrial repositories. The term "context engineering" is not properly standardized as of October 2025, but this thesis uses it as an umbrella term for all context-related techniques such as prompt engineering, augmentation methods and repository-specific fine-tuning. By comparing different methods, this literature review aims to gather information about practical strategies for deploying and improving the performance of LLMs in enterprise software development environments.

The key questions addressed in this thesis are:

1) What are the most effective methods to provide domain-specific context to enhance software development related performance of LLMs?

2) What are the differences in performance and practicality between these methods?

3) Are these methods adoptable in industrial environments?

1.3 Scope and Structure of this Thesis

This thesis conducts a broad overview of the selected methods and their effects on performance of LLMs with software development tasks. This thesis is structured around the context engineering taxonomy by Mei et al. [3]. Additionally, this thesis focuses on techniques that can be implemented without developing or adjusting the parameters of the LLM itself.

First, in Chapter 2, the concepts of context engineering are defined. Next, in Chapter 3, the AI-assisted software development problems related to context engineering are introduced. Chapter 4 analyzes the methods starting with the first area "context generation and retrieval" based on the taxonomy by Mei et al [3]. Chapter 5 continues by analyzing the third "context management" area. The second area of taxonomy i.e. "context processing" is left out for the most part as it concentrates on evolving the LLMs themselves, whereas areas 1 and 3 focus on enhancing existing models. It is not worthwhile for all companies to develop their own LLMs, as the pre-trained models such as the

performant DeepSeek version 3 are available as open-source [7]. The results of the literature review will be discussed and analyzed in Chapter 6. Chapter 7 concludes this thesis.

2 Defining Context Engineering

Context engineering (CE) refers to the systematic design, retrieval, organization and maintenance of information that enables LLMs to reason effectively inside of dynamic repositories and helps to stay within the boundaries of their context windows [3], [8], [9]. CE extends traditional prompt engineering by treating context as an engineering artifact rather than as a single-use prompt. Grove [10] from OpenAI believes that contextual information as an artifact will be the future direction of agentic AI systems.

The field of context engineering is fragmented, and it only began to be defined in 2025. This thesis will utilize the taxonomy introduced by Mei et al. [3] that divides CE into three foundational components; context generation and retrieval, context processing and context management. The taxonomy is illustrated in Figure 1.

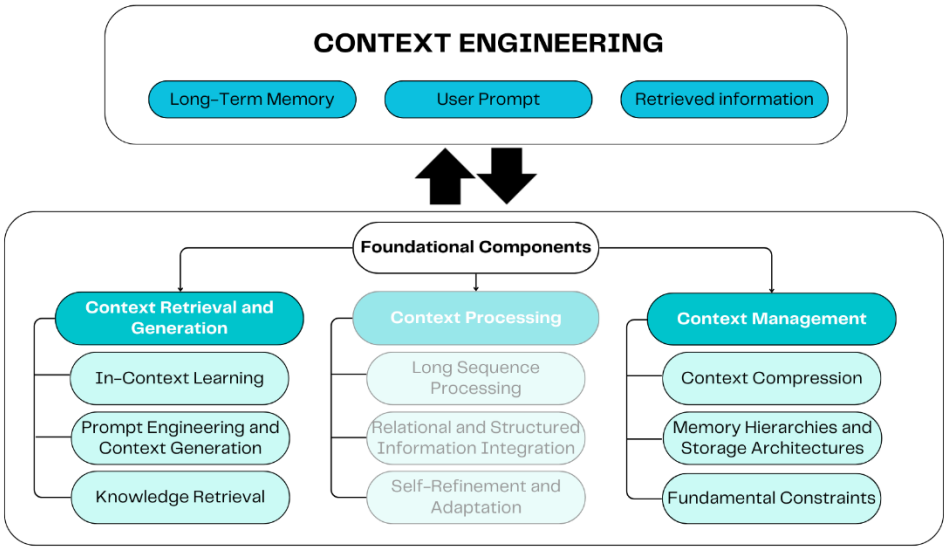

Figure 1 – Context engineering and its foundational components based on Mei et al. [3]. Context Processing, the middle of the three foundational components, is left out of scope of this thesis.

The *Context retrieval and generation* – component of the taxonomy includes techniques that are responsible for the acquisition of knowledge and guiding the model. In-context learning is a vital capability of LLMs that enables efficient prompt engineering and learning during inference [3], [6]. *Knowledge retrieval* helps the model by returning input snippets, function signatures, or documentation relevant to the task. Knowledge retrieval methods are vast, but the most researched technique is the *retrieval-augmented generation* (RAG) [9]. RAG extends the effective context by dynamically fetching relevant code fragments and documentation from a pool of information such as

a repository [11], [12]. The returned context is given to the model prompt to enhance outputs that align with existing patterns and conventions.

Context processing focuses on refining and optimizing the retrieved or generated context to enhance its usability for LLMs. Context processing addresses challenges such as redundancy, irrelevance and excessive length of documents. Techniques in the context processing category include prioritization via relevance scoring and self-refinement loops where the model iteratively evaluates and improves the context based on feedback mechanisms like execution verification or error analysis [3]. In many applications, the context processing methods improve performance and memory handling capabilities of the model by a significant margin. However, context processing methods come at a significant cost in the difficulty of adoption. For example, in tasks of managing complex codebases, context processing can enable the filtering of noisy dependencies, syntax trees or documentation to produce more precise and integrable code outputs. For instance, approaches like FlashAttention [13] for efficient computation or Reflexion [14] for reflective adaptation help mitigate the "lost-in-the-middle" phenomenon in long sequences [15], ensuring that the LLM remains alert of critical repository-specific details during generation tasks. One state-of-the-art performance wielding implementation is QWENLONG-CPRS, which can reduce dynamic context sizes by up to 290 times [16]. QWENLONG-CPRS is remarkably performant, but it requires heavy engineering inside of the LLM, such as using Qwen2 as the base model, making it very rigid and difficult to utilize in real-world applications.

The last category in the taxonomy, *context management*, stands for the strategies for storing, organizing, and persisting context over time. This ensures seamless handling of contextual information across multiple interactions without overwhelming the finite context window of the model. Context management involves hierarchical memory systems, such as short-term episodic buffers for immediate recall and long-term semantic storage via external databases or knowledge graphs. The memory systems are often complemented with eviction policies like First-In-First-Out (FIFO) or diversity-based removal to maintain memory efficiency while continuously storing fresh knowledge [17]. MemGPT [17] is a well-known memory system LLM that emulates operating system paging for infinite context. *Cache-augmented generation* (CAG) is another strategy to augment context by searching previously seen information to return relevant context for the LLM improving the model outputs [18], [19].

There are also different layered multi-agent frameworks that play a key role by distributing context among specialized agents. They can operate by dividing the task in a tree-like manner where one agent calls another sequentially.

They have the ability to also work simultaneously, for example where one model is retrieving dependencies while another manages state consistency in iterative coding workflows [3].

The foundational areas of context engineering, and a few examples of implementations, will be examined later in Chapters 4 and 5. The first and third areas, context generation and retrieval and context management respectively, will be discussed in more detail as they more adoptable in industrial environments [3]. The focus will be on methods and implementations that are useful in practice instead of only discussing theoretical frameworks that are not applicable in real world environments.

3 Relevant Issues in AI-assisted Software Development

The context window is the working memory of LLMs. The context windows of LLMs include a fixed number of tokens. Different models have varying number of tokens available, ranging up to millions with the most modern models. The limited number of tokens pose limits for the memory and context understanding of the model. When a context window is exceeded, the model begins to lose information and its performance degrades drastically [8], [20].

The intrinsic issue with LLMs is that they only possess static information that has been included in their training, which leads to hallucinations when the model attempts to answer a question that it does not have information of [20]. Related issue is when information evolves. When information changes after the model has been trained, the information in the parameters of the model become outdated. This can lead to hallucinations and misinformative outputs where the model confidently produces inaccurate outputs based on the old information in the parameters. These issues can be improved by runtime context retrieval methods, such as RAG [11].

A central problem is that simply increasing the context window size with more tokens does not consistently help with model performance [15]. Increasing the number of tokens quadratically grows both memory usage and computing power demands making the model require more powerful hardware and bigger data centers to operate [13]. Furthermore, LLMs often have difficulty maintaining attention on the middle sections of long inputs, which is a limitation attributed to their U-shaped attention pattern [15]. This pattern causes the model to emphasize the start and end of the context, thereby weakening the potential benefits of longer context windows. Due to the vast number of models and external context management techniques, indicating general rules for too long contexts is not feasible. One token is generally agreed to be approximately equal to 0.75 English words. Therefore, modern long-context models with context windows of over hundreds of thousands can efficiently operate on inputs such as entire codebases or books. However, the lost-in-the-middle effect is amplified the longer the input is [15]. In addition, many techniques, such as context compression, can shrink the token usage below the generic ratio of three tokens to four English words.

Another limitation of LLMs is the lack of system-level and domain-specific knowledge. Without extensive fine-tuning, LLMs are usually not trained on use-case specific information, such as embedded software or safety-critical programming standards, which are vital for industrial software development

[20], [21]. As a result, and without proper context engineering, LLMs may generate code that violates conventions, safety constraints and hardware limitations. One aspect that amplifies difficulties is that most proprietary agentic systems are not open about which context engineering techniques the system utilizes, and which would further enhance performance. The gap between pretraining and specialized industrial knowledge reduces the flexible applicability of general LLMs in high specificity domains such as embedded or real-time systems. However, the knowledge limitations are mitigated by the first section of the taxonomy as depicted in Figure 1, context retrieval and generation.

4 Context Retrieval and Generation

Context retrieval and generation is one of the three foundational components of context engineering. It is responsible for systematically retrieving all of the relevant information for the LLM for example [3]. This Chapter consists of three primary methods as discussed by [3]: ICL, prompt engineering and external knowledge retrieval. However, this thesis does not cover all aspects of the taxonomy.

4.1 In-Context Learning

ICL represents a revolutionary emergent ability of LLMs, enabling them to adapt to new tasks during inference without requiring any updates to their parameter weights [22], [23]. The ICL capability, which is typically observed in models exceeding a scale of well over billion parameters, allows LLMs to leverage patterns learned during pre-training to perform tasks based solely on the provided prompt context [23]. ICL is on the borderline of context generation and retrieval and context processing components, but it is vital for many techniques like prompt engineering. Therefore, it is analysed in this thesis.

The field of ICL has become very active within the LLM development overall. ICL is characterized by overlapping, and sometimes competing theories, that seek to explain its mechanisms and effectiveness. For example, some papers indicate that ICL mimics implicit gradient descent within the forward pass of the model and others view it an aspect of meta-learning or Bayesian inference [4], [22], [23]. The ability to in-context learn differs from model to model and the influencing factors include model architecture, training methodologies, dataset for training, and the scale of the model [22], [23]. Certain empirical studies indicate that models trained on diverse and high-quality datasets boost the ICL abilities of the model, in contrast to those trained with poor or little training data that lead to worse generalization [23].

The main contribution of ICL is that it enables LLMs to adapt to context-specific requirements during inference by using relevant knowledge within prompt. To utilize advantages of ICL to the best extent, the knowledge in the prompt should include everything the model could benefit from, such as task instructions, domain-specific guidelines and examples of the tasks. ICL stands in contrast to traditional machine learning paradigms, which generally rely on gradient-based optimization of parameters through training and fine-tuning with labeled datasets to adjust model weights. Instead of parameter optimization, ICL more effectively exploits the pre-trained knowledge embedded in the model parameters via techniques such as zero-shot, one-

shot, or few-shot prompting, which enable flexible and efficient task adaptation without computationally intensive retraining of the model [22]. In the domain of software development, especially with large-scale codebases, ICL can be very valuable. Dynamic contextual understanding is essential for producing integrable code that aligns with repository structures, coding standards, and functional requirement [24]. By incorporating repository-specific details and examples into prompts, ICL empowers LLMs to generate more reliable outputs, such as functions, or even entire modules, making it important for AI-assisted software development considerations.

Traditional learning in the field of machine learning, such as neural networks including LLMs, refers to modifying parameter weights with backpropagation and different optimization algorithms. However, ICL differs from this traditional view by operating entirely at inference time, completely without altering the parameters of the model [22]. For this reason, most agentic AI systems, such as GitHub Copilot [1], use underlying models that have strong ICL capabilities. The exact conditions under which ICL emerges is an ongoing debate, with multiple proposed theories. Some theories explain ICL is the ability of the model to dynamically map input spaces to its weight structure effectively simulating task-specific adaptations [4]. In practical environments, such as code generation tasks, ICL enables the model to utilize prompt-level information, such as static instruction files containing guidelines, industry-specific conventions and optional examples. Recent comprehensive surveys underscore the importance of ICL in enhancing the performance of modern LLMs across many different tasks, such as code generation, where it enables models to achieve state-of-the-art results by synthesizing pre-training knowledge with dynamic context information during inference [3], [20]. ICL forms the basis for context engineering methods, and many challenges, such as sensitivity to prompt quality, token limitations, remain to be solved. ICL also complements retrieval methods as the information in the prompts can be dynamically augmented to be the most useful in each task individually via methods like RAG [11].

4.1.1 Zero-shot

Zero-shot in-context learning represents the most basic variant of ICL, where the LLM is prompted to perform a task without any prior examples or demonstrations. In zero-shot ICL, the model draws exclusively from its pre-trained knowledge to interpret and execute the instructions embedded in the prompt. The specific instructions given to the model are aspects of prompt engineering, and they can come in many forms such as in a static pre-prompt instruction file or in some other template. For instance, in AI-assisted software development tasks, a zero-shot prompt might instruct the LLM to “Write C++ code that computes the matrix multiplication ensuring that the

result is diagonalizable" without providing further examples what the task means or how to conduct the task. The prompt could also include the general instructions in a file where the desired specifics, such as what intermediate steps to show and what symbols to use, would be located. The zero-shot ICL prompt utilizes the emergent ability of the LLM to generalize information enabling them to infer the natural language from the prompt alone [6]. Unlike few-shot ICL, which incorporates examples to guide the model, zero-shot learning minimizes the prompt complexity and token usage. This makes zero-shot ICL computationally more efficient and straightforward compared to few-shot. In addition, zero-shot is considerably easier to set up for the end-user, as it doesn't require the labor-intensive process of manually assembling examples for the model or setting up retrieval tools to augment the prompt automatically [3].

To address the challenges with manual or limited information, augmentations such as demonstration augmentation and others discussed later, have been proposed. Demonstration augmentations means that the past zero-shot predictions of the model serve as implicit examples for subsequent inferences, improving consistency with only a small effect to the token usage [25]. In large-scale software development, zero-shot ICL serves as a baseline for building more advanced strategies such as Repo-Level Prompt Generator (RLPG) [26]. Zero-shot is the baseline for all models capable of ICL. It indicates that the model does not have any static or dynamically augmented examples of the task it tries to conduct. While zero-shot ICL is favorable for its simplicity and speed, methods providing context or examples could often improve performance [6], [25].

One example of a zero-shot technique implementation is the Demonstration Augmentation for Zero-shot In-context Learning (DAIL) [25], which overcomes the limitations of pure zero-shot ICL by using the historical predictions of the LLM as possible examples for future tasks. DAIL operates without external data or additional generative steps as it only stores the inputs and outputs of the model locally into memory [25]. It constantly keeps a fixed-size memory to store input-output pairs from prior inferences. The process initially starts with an empty memory, only using normal zero-shot prompting for the first query. DAIL then selects a few demonstrations from the bank on later inferences using a combined scoring mechanism based on a selection score. The selection score can be calculated in many ways, for example with best matching 25 (BM25) for textual similarity or TopK cosine similarity via embeddings. After the inference, the current input and the output are added to the memory bank, and if capacity is exceeded, some samples are deleted from the memory using strategies like First-In-First-Out (FIFO) or diversity-based removal to upkeep the memory with relevant knowledge.

Self-augmentation via DAIL can improve consistency and performance by dynamically turning zero-shot prompting into few-shot prompting after a few queries [25]. DAIL achieves higher performance compared to baseline without increasing prompt complexity and with little computational increase beyond standard zero-shot ICL. DAIL has advantages, including reduced sensitivity to demonstration quality, no need for human-annotated examples, and inference speeds comparable to zero-shot. DAIL proves to be very flexible and quick, but its performance may not be comparable to more advanced methods [25].

4.1.2 One-shot and Few-shot

One-shot and *few-shot* ICL extend the baseline of zero-shot ICL techniques by providing demonstrations or examples to the model within the prompt. One-shot ICL means that the prompt includes one illustration of the task in the prompt and few-shot means that the prompt includes two or more examples. It is shown that providing the model with some examples within the prompt can increase the performance of the model [23].

A key technique for improving one-shot and few-shot ICL, particularly in code generation tasks, is *Large Language Model-Aware In-Context Learning* (LAIL) [6], which introduces a demonstration retriever trained to work with the preferences of the generator LLM. LAIL uses a LLM to estimate and label candidate examples to maximize the probability of augmenting the prompt with the most relevant examples. The process involves three stages. First, for each training example, a retriever filters a small candidate set and the LLM calculates probabilities for the ground-truth program given by each candidate and the query. Best candidates among the group are marked positive and bottom-scoring candidates as negative, creating a labelled dataset. Second, a retriever is trained to rank examples by cosine similarity, incorporating hard negatives for robustness. Finally, during inference, the retriever selects best scoring examples based on the previously calculated embeddings, which are added into the prompt for code generation.

The results of ICL without external data retrieval in code generation tasks is observed to be below the state-of-the-art implementations of RAG techniques as the LLM relies only on the instructions and examples it is given without further external knowledge [6], [11]. The quality of outputs ranges heavily from excellent all the way to poor, unusable, code [6]. This proves that ICL is capable of generating high quality code, but the challenges remain with domain specific information and providing the model with quality prompt. It is shown that good instructions improve both zero-shot and few-shot ICL [25]. There exists many additional techniques solving the issue of

providing quality prompts with contextual information to the LLM in addition to RLPG [26], LAIL [6] and DAIL [25].

4.2 Prompt Engineering

Prompt engineering is the discipline of engineering the optimal textual input to guide LLMs towards the desired output without altering the parameters of the model [27]. While ICL provides the underlying capability for models to adapt to new information during inference, prompt engineering is one practical application of this capability. Prompt engineering is a broad discipline that consists of dozens of distinct approaches. Most techniques aim at making prompting a systematic way to develop more reliable performance across queries [27]. For example, system prompts, role-playing and chain-of-thought aim at making the LLM produce more reliable outputs across queries.

Unlike casual interaction with LLMs, professional prompt engineering aims at treating the prompt as an engineering artifact [3], [10]. This makes it possible to make systematic progress in the performance of prompt engineering as the prompts are carefully crafted and saved rather than used as one-time inputs. Such systematic prompting is often achieved through a *system prompt* or *static instruction file* that refer to a pre-defined context file. The most important and recurring information could then be defined in these files to make sure that the LLM is always aware of the most important context. In software development environments, this could mean that style conventions, available resources and other important context is always included in the prompts. This increases the performance of the model at producing code that aligns with the domain-specificity within different organizations [27].

Role-playing is an approach that is often used within system prompt files. Role-playing means that the system prompt includes information about the role of the LLM. For example, the system prompt could have section that explicitly indicates that the role of the LLM is to be an embedded system engineer and to produce code that aligns with the repository conventions. *Chain-of-thought* on the other hand is a niche technique that guides the model to articulate its reasoning by requesting the model to operate by chain-of-thought [3], [27]. This can enhance performance and is often implemented in agentic AI systems, such as GitHub Copilot [1].

4.3 Retrieval-Augmented Code Generation

At its core, RAG generally operates through a two-stage process. The process involves a retriever and a generator. During inference, the query or the

prompt is given to both stages simultaneously. The generator then waits for the retriever to return relevant documents for a more comprehensive contextual understanding. The generator is then able to utilize the initial prompt accompanied by the retrieved documents as additional information for the given generation task. This enables the LLM to generate outputs with better accuracy, but it does not automatically lead to better performance [28], [29]. The retriever must be carefully engineered to return relevant documents for the specific task, and the generator must be able to infer the knowledge for it to be useful. The returned documents must be low-noise and low in token usage, or the retriever might introduce irrelevant noise that decreases performance [9]. RAG also increases latency and requires additional computation for pre-computing and for the retrieval process [28], [29]. Additionally, the decoupling of retrieval and generation allows for better scalability, as the knowledge base can be updated independently of the model. This makes RAG particularly suitable for environments in which information evolves, such as code repositories. RAG techniques can be categorized into Graph-based and Non-Graph-based strategies [9].

Figure 2 illustrates a high-level process workflow of a LLM system utilizing RAG. First, the repository is often turned into a vector database that can be efficiently queried. Then the retriever queries the database based on the given prompt and returns useful information for the generator. This often helps LLMs to produce more reliable and accurate outputs [9].

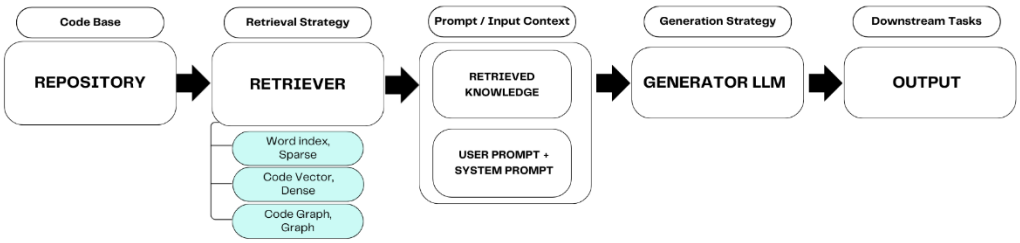

Figure 2 – Illustration of RACG process.

One relevant method to augment context for the LLM in software development environments is *Retrieval-Augmented Code Generation* (RACG) [9], which is an evolving field inside of general RAG techniques. RACG specifically concentrates on code generation tasks whereas RAG can be used in generation tasks in general. As of 2025, many RACG techniques represent the most advanced strategies among the context engineering field, producing high performance in code related tasks [9]. RAG and RACG are used interchangeably in this thesis, as the focus is centered on code generation tasks in general. RAG builds directly on top of the ICL capabilities of modern LLMs in order to learn new patterns during inference [25]. The examples in the following text RepoCoder [30] and GraphCoder [31] were picked from a large

pool of potential implementations of such methods as they are well known within the industry and their performance has been widely benchmarked. This makes it easier to compare the differences between the methodologies and also increases reliability of results as there is more data.

RAG is used to integrate LLMs with large external knowledge sources, such as code repositories or manuals, designed to overcome the inherent constraints of context window sizes [12], [31]. Additionally, RAG tries to overcome the limitation of the fact that pre-training does not generally prepare for specific downstream tasks. In addition, techniques such as fine-tuning need constant maintenance to keep up with constantly evolving information, which could benefit from dynamic information augmentation [31]. By dynamically retrieving information during inference from vast repositories or databases and injecting that information into the prompt of the model, RAG enables contextually rich responses from the model without requiring the model to store all the information internally into its parameters.

Unlike RAG, methods such as CAG and memory systems are intended to store and utilize information that the model has already seen previously. These context management techniques can complement many RAG techniques, and they will be discussed further in Chapter 5. Many RAG techniques have quickly achieved the state-of-the-art status among the context engineering strategies [28]. Such techniques enable models to address real-world issues in knowledge-intensive tasks such as summarization, code generation and answering specific questions [9].

4.3.1 Non-Graph-Based Retrieval-Augmented Generation

Non-graph-based RAG refers to RAG methods in general that operate searches via lexical or semantic similarity as explained by Tao et al. [9]. The authors also identified that non-graph RAG implementations often treat codebases as flat sequences of text lacking explicit structure. In contrast, graph-based implementations create knowledge graphs that rely on the structural dynamic of code repositories. In RACG tasks, the non-graph-based RAG methods retrieve relevant documents based on either lexical similarity or semantic similarity. *Lexical similarity* is a simple technique trivially comparing occurrences of similar words inside of the analyzed documents and calculating the similarity score based on the count of similar words. Lexical similarity calculation tools include, for example, well-known BM25 and Jaccard similarity. On the other hand, *semantic similarity* is about comparing the meaning of two documents with a more complex operation. Semantic similarity is calculated with the embeddings of words inside of a vector database and by comparing distances between the vectors. The usage of semantic similarity is preferred over lexical similarity in more advanced solutions [9].

Tools utilizing semantic similarity include methods such as UniXcoder [32] and CodeBERT [33]. In general, lexical similarity-based retrieval is referred to as *sparse retrieval* and semantic similarity-based retrieval as *dense retrieval*.

One widely recognized foundational non-graph-based RACG method is RepoCoder [30]. RepoCoder is an iterative retrieval implementation, where retrieval and generation steps alternate. The process progressively refines the context used for code generation. RepoCoder executes multiple retrieval rounds and each round is informed by the intermediate outputs of the model instead of relying on a single retrieval. This iterative process enables the retrieval module to narrow down the most relevant code fragments across large repositories, improving both accuracy and contextual consistency. Results of the Repocoder prove that it provides an improvement of 10%–20% in exact match benchmarks [30].

The architecture of Repocoder integrates a retriever that leverages dense embeddings generated via pretrained models such as CodeBERT or UniXcoder, which transform code snippets into semantically meaningful vector representations [30]. This convention of generating vectorized representations of words into a vector database is common in many RAG techniques [9]. Similarity in RepoCoder is computed through cosine distance, allowing the retriever to capture deep semantic relationships rather than simple keyword overlap. The generator, such as a transformer-based LLM Codex, then uses the retrieved context to produce code that aligns with repository-specific conventions.

The main advantages of non-graph-based RACG systems like RepoCoder are efficiency, language-agnostic design, and ease of scalability as they can be deployed across diverse environments without the need for heavy preprocessing or static code analysis [9]. In addition, because these methods operate on word embeddings or textual features rather than abstract syntax trees, they are compatible with many programming languages and model architectures.

However, the absence of structural modelling also introduces limitations. Non-graph-based RAG often struggles with deep dependency reasoning, such as understanding inter-file relationships or inheritance hierarchies [9], [12]. As a result, while they perform well for single-function or file-level completion tasks, their effectiveness decreases for repository-level reasoning where code entities depend heavily on one another [12]. Nevertheless, their simplicity, efficiency and generality make them useful for a practical RACG solution.

4.3.2 Graph-Based Retrieval-Augmented Generation

As described by Tao et al. [9], *graph-based RAG* extends standard RAG techniques by introducing explicit graph representations. The authors explain that the graph representations capture structural and semantic relationships between code entities, empowering deeper understanding of the context compared to simple textual similarity. Many graph-based RAG implementations construct the graphs in their own unique ways, but often the graphs follow the following generic guidelines: nodes represent elements such as functions, classes, or modules, and edges represent relationships such as containment, inheritance, invocation, or data and control flow [9], [31]. Retrieval with graph based RAG is rather underexplored and it is currently performed in dozens of ways, some more sophisticated than others [9]. These approaches enhance the model at understanding code repositories as an interconnected system, leading to improved accuracy in tasks involving long-range dependencies, such as cross-file code completion. However, these benefits are repository dependent. In large and homogenous repositories, the added structural depth may yield diminishing returns compared to long-context models, while in diverse or multilingual codebases, graphs offer much better cross-lingual alignment and generalization [9]. For example, Du et al. [12] introduced CodeGRAG that offers superior performance in multi-language repositories compared to traditional non-graph RAG. The proof-of-concept results were conducted by using Python and C++ in the HumanEval-X benchmark tool. The tests show that the knowledge graph itself increases performance only by 2%–8%, but the performance could be increased by over 100% with prompting techniques that help the LLM to understand the graph better.

Tao et al. [9] recognize that the immediate downsides of graph-based RAG compared to non-graph based RAG are the maintenance, preprocessing and composition requirements of generating the structured graph. However, the upside of providing structural understanding of the repository for the model can be substantial in optimal environments [9]. For instance, by modelling relationships like data flows and control dependencies, graphs mitigate common issues in non-graph RAG, such as overlooking implicit connections between code entities, which results in more coherent and contextually relevant generation [12], [31].

One example of a graph-based RACG system is GraphCoder [31]. The inventors of GraphCoder acknowledge that the standard non-graph RAG inherently misses out on many crucial structural aspects about the relationships between code entities by only retrieving snippets based on sequentially created vector embeddings. GraphCoder tries to overcome these limitations by creating a *code context graph* (CCG) of the source code and using that in the

retrieval to catch important structures within the repository. CCG is the superimposition of three separate graphs that each represent different aspects of the code: *control flow graph*, *control dependence graph*, and *data dependence graph*. The retrieval is done by coarse-to-fine steps and the effectiveness of CCG is achieved by replacing sequential representation with structured representation [31]. In addition, the authors state that the effectiveness comes from adopting "decay-with-distance" to prioritize code elements that are semantically and topologically close. The authors also mention that CCG is language-agnostic, making GraphCoder viable with any programming language.

The authors of GraphCoder [31] conducted proof-of-concept benchmarks by using Python and Java in the RepoEval-Updated benchmarking tool. The benchmarking has been done thoroughly with comparisons between baseline LLM, non-graph RAG, RepoCoder and GraphCoder. The results show that non-graph RAG enhances performance by approximately 50% over baseline LLM performance. GraphCoder enhances performance by additional 5%–15% over non-graph RAG. The performance of RepoCoder falls between non-graph RAG and GraphCoder in this benchmark with three iterations used. This is interesting as it is arguably stronger performance boost compared to the exact match improvement of 20% represented in Repocoder's own paper [30].

4.4 Repository-Level Software Development Considerations

In the context of software development in large repositories, zero shot ICL is particularly useful due to it being computationally light. It is not outstanding technique alone, but it is the backbone to enable developers to rapidly improve LLM performance by using more advanced techniques such as system prompting and RAG [3]. ICL is agile without retrieval latency with only moderate context-window burden. In addition, there is some evidence of ICL providing better generalization compared to fine tuning [4]. One example of state-of-the-art technology that enables usage of ICL in code-generation tasks as of 2025 is GitHub Copilot [1]. GitHub copilot enables developers to construct user-specific system prompt files that can contain static information that is injected into every prompt enabling the model to produce enterprise-specific code [1]. The prompt with its instructions enables the underlying model to produce high-quality and repository-specific code and the developers can customize the instructions to their specific needs.

The effectiveness of zero-shot ICL stems from the prompt engineering used to craft the input. Prompt engineering is a fast-growing field of AI that is

crucial to understand to be able to properly utilize LLM systems. There are dozens of useful prompt engineering techniques for software development needs, but it is important to pick a few and optimize the LLM system through them. However, even with adequate prompt engineering, the performance of the model is inherently limited by its pre-training as it affects its knowledge and efficiency with context sizes [3], [15]. In domains with sparse and scattered information, such as embedded systems codebases, zero-shot on its own may violate constraints or produce inefficient code as the pre-training has not included enough data for the context [3]. Context window limitations should also be considered as overly verbose prompts can lead to degraded attention in long sequences [15].

While ICL and prompt engineering form the basis for efficient LLM operation, RACG techniques enable the model to adapt to niche environments [3], [9]. Without augmentation during inference, LLMs would be inherently very limited in terms of exact and up-to-date information. RAG techniques empower LLMs to understand contexts to the extent that they enable the generation of accurate, relevant, and up-to-date responses in repository-level environments [9]. RAG is especially useful in niche environments, where the model benefits the most of augmented context. RAG could, for example, search through enterprise-specific knowledge bases, such as enterprise-specific wikis, to retrieve relevant information that the model could not otherwise access. However, the LLM system can go even further. By incorporating context management techniques, the system can process and store information more efficiently, reducing the computational overhead and mitigating the fundamental constraints [3].

5 Context Management

While Chapter 4 on context retrieval and generation focuses on generating and gathering information, context management focuses on the efficient organization, storage, and utilization of contextual information across queries [3]. Context management enables LLMs to overcome the inherent limitations of the transformer architecture, such as lost-in-the-middle related problems and fixed context windows, which hinder the performance and scalability of LLMs [3], [15]. In AI-assisted software development, these issues are particularly present, as code generation often involves iterative tasks across large repositories, requiring continuous access to prior interactions, dependencies, and domain-specific knowledge [3]. Common techniques used in context management are memory hierarchies for layered information access, storage architectures and compression techniques to maximize the information density in the stored data to maintain better accessibility and coherence [3].

5.1 Memory Hierarchies and Storage Architectures

Memory hierarchies organize and store contextual information into layered structures, mimicking human cognition and computational systems to handle long-term contexts more efficiently [3], [34]. This allows LLMs to prioritize recent or relevant data in short-term layers while storing broader knowledge in long-term layers, reducing issues with overflow and improving reasoning over extended contexts [3].

Storage architectures provide frameworks often paired with memory hierarchies. Storage architectures can be implemented in very different ways, but for example some draw the idea from operating system designs to manage memory as a virtual resource [3], [17]. These memory systems enable LLMs to handle virtually infinite contexts by storing data into external stores when the context window is full, which supports agentic behaviors in software development where models act autonomously over multiple or longer sessions [3], [17].

A foundational and widely used method utilizing sophisticated memory implementation is MemGPT [3], [17]. It is one of the methods drawing from operating system architecture. MemGPT extends the context window of the model by dynamically paging relevant information into the active prompt, similarly to how operating systems manage virtual memory between RAM and hard-disk [17]. This technique allows for effectively infinite context where the information can be retrieved from the external memory during inference [17]. MemGPT writes the external database itself using dedicated functions, which is made possible by using a system prompt file where the

functions available and memory constraints are introduced to the LLM [17]. The system prompt file makes the LLM aware of its own memory constraints, and at the same time enables it to manage the memory itself via read and write operations to the memory [17]. The authors of MemGPT prove that it improves accuracy by up to 150% in certain situations, but the results are not directly generalized in software engineering environments.

More recent implementation of data storage enhanced LLM by He et al. [34] is the Consolidated Associative Memory Enhanced Long Transformer (CAMELoT), which enables the system to tackle long contexts by mimicking human memory consolidation. The authors explain that CAMELoT works by creating associative memory slots that hold associations between queries and representations. The paper shows that CAMELoT is especially interesting as it implements an independent memory module from the model that is model agnostic, enabling the usage of the memory module with different LLMs in various environments. The paper proves that CAMELoT can reduce perplexity by up to 30% in Arxiv benchmarks, but that results do not implicitly increase software development performance linearly.

5.2 Context Compression Techniques

The survey by Mei et al. [3] defines *context compression* as techniques enabling LLMs to operate on longer contexts by reducing the computational and memory needs while preserving the integrity of information. There are vast implementations to achieve context compression, such as memory-augmented approaches and hierarchical caching systems [3]. The survey also acknowledges that certain context compression techniques, such as QWENLONG-CPRS [16], are used within the models themselves, but those methods belong in the context processing foundational component and are not further discussed in this thesis. Context compression methods in this thesis are external of the model itself and can be implemented as external systems. Even CAMELoT can be argued to be a context compression technique, as it creates an external memory by compressing information via arithmetic averaging [34]. The Figure 3 illustrates the general idea of compression techniques. The central idea is that verbose parts of natural language can be completely left out while keeping the informative parts of the prompt. Compression techniques can also be used in other parts of the system in addition to direct prompt compression. For example, the input to memory systems is often compressed to fit more data [3].

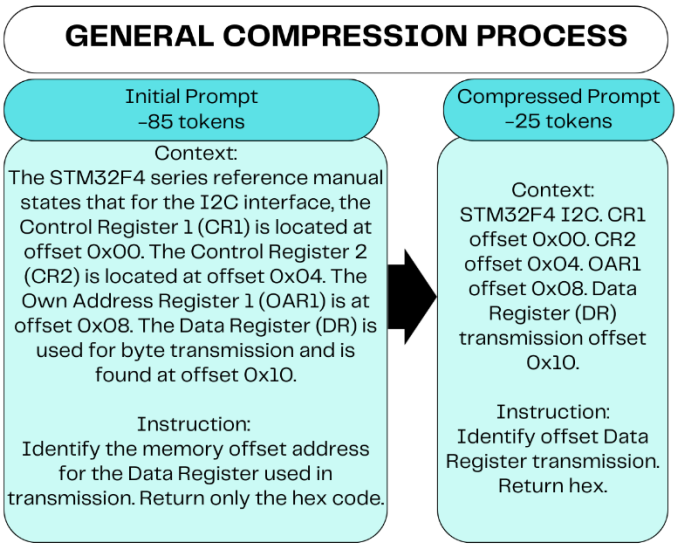

Figure 3 – Illustration of the general idea of compression techniques. It achieves reducing memory usage while preserving almost all information.

LLMLingua, introduced by Jiang et al. [35] is a lightweight implementation that utilizes context compression. The authors state that it is a pre-processing-stage approach that modifies the prompt by compressing the redundant token-level information out of the prompt. The paper indicates that LLMLingua detects important tokens by their high perplexity compared to average tokens. LLMLingua does more than just context compression though. It consists of three modules that are illustrated in Figure 4. The budget controller is responsible for determining what parts of the input are necessary for the model to process. The iterative token-level prompt compression is the relevant component here that is capable of compressing the relevant context into a more compact representation without losing relevant knowledge in the process. The distribution alignment module helps LLMLingua to generalize better on a wide range of LLMs. The paper proves that LLMLingua is capable of reducing the token usage in the prompt up to 20 times smaller while only losing less than two percent of the relevant information. The process workflow of LLMLingua is illustrated in Figure 4.

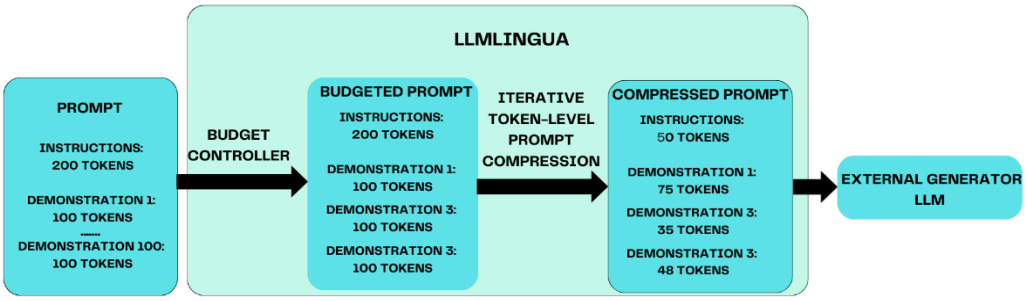

Figure 4 – Illustration of the LLMLingua process based on [35].

5.3 Repository-Level Software Development Considerations

In repository-level software development, many context management techniques are crucial as they enable retaining knowledge about project-specific architecture, enterprise-specific information and more over traditional context window limitations [3]. For instance, with context-preserving systems, such as CAMELoT, LLMs can recall that a repository follows specific naming conventions or hardware safety requirements without manually reintroducing this information in every prompt. This mechanism effectively acts as a dynamic memory-based prompt augmentation process, which reduces the need for manual repetition. Context management enables the model to process longer documents, contexts and to retain such information longer to improve long-term performance [3].

6 Results

This chapter presents the results of the literature review by synthesizing the findings of different aspects of the two foundational areas of context engineering, context generation and retrieval and context management. This Chapter aims to answer the research questions presented in Section 1.2. The results in this chapter represent a qualitative synthesis of the findings presented in the reviewed papers. As there is currently no standardized benchmark comparing all these methods simultaneously, this analysis relies on a comparative review of the metrics reported in the source literature.

6.1 Effectiveness of Context Generation, Retrieval and Management Methods

The literature reviewed in this thesis indicates that the most effective methods for providing context, especially domain-specific context, to large language models in software development tasks are retrieval based systems such as RAG [3], [9]. However, the retrieval methods can still be accompanied by many other techniques by utilizing them simultaneously to further increase performance. Zero-shot ICL is an efficient baseline but limited by performance in environments requiring repository-specific knowledge. Few-shot and augmented ICL improve performance at the cost of minimally increased token usage and reduced scalability. Retrieval-based approaches consistently outperform standalone prompting as they introduce external, task-relevant information that the model cannot store in its parameters [9].

Non-graph-based RACG provides strong improvements among RAG methodologies through its similarity search, and it is effective for large modular codebases. On the other hand, the more advanced graph-based RACG further enhances performance by capturing control flow, dependencies, and structural relationships. It enables the model to operate reliably in complex repositories written with multiple different programming languages [12]. These gains are particularly relevant for tasks requiring cross-file reasoning and staying within architectural constraints, but comes at a substantial cost in terms of complexity to take into use [12].

Within context management, methods based on memory hierarchies, external storage, and prompt compression are the central techniques for preserving relevant contextual information without exceeding context window limits. Lightweight compression techniques, such as LLMLingua, enable substantial reductions in token usage, which often enhances performance. On the other hand, memory-based systems allow models to retain repository-specific patterns over long interactions. Such memory-based solutions are

sometimes overshadowed by prompt engineering techniques such as static instruction files that also include similar knowledge. The performance of different methods based on the various benchmarks is illustrated in Table 1.

6.2 Performance and Practicality of the Methods

Across the papers reviewed in this thesis, retrieval-augmented techniques demonstrate the highest performance for repository-level software development tasks [9]. Non-graph-based RACG achieves substantial performance gains with moderate engineering effort, whereas graph-based RACG offers the strongest overall performance by incorporating structural information that LLMs do not infer from textual prompts alone. However, graph-based approaches require more extensive preprocessing and maintenance, which reduces their practicality and adoptability.

Pure ICL techniques utilizing prompt engineering express variable results. Their performance depends heavily on prompt quality, and they become less effective as repository complexity increases. Although augmented or few-shot prompting can mitigate this variability, these approaches can bring growing token overhead. In contrast, retrieval systems scale more effectively on larger repositories because they offload contextual information into external storages rather than embedding the information directly into prompts.

Context management methods can bring performance boost by enabling consistent recall of repository-specific conventions and reducing token requirements. Compression-based preprocessing offers a practical way to increase context density with minimal overhead. Memory-based architectures provide stronger long-term consistency but can require more complex integration.

Overall, the literature identifies a trade-off between performance and implementation complexity. The most advanced techniques, such as graph-based RACG, often bring bigger performance boosts compared to more trivial methods like few-shot learning. Non-graph-based RACG and lightweight prompt engineering techniques provide the best balance of effectiveness and practicality as the most sophisticated methods deliver higher performance at the cost of too high operational complexity.

The results described in Table 1 summarize the most relevant pieces of information of the overall results. The Category column illustrates what context engineering area each row is about. The column of relative performance is about how much the method can improve the LLM performance in producing better outputs in software engineering environments. Token cost resembles the impact on the internal context window load of the model. Complexity

column is about how difficult it is to adopt the method in general. Best environments column expresses the optimal environment to achieve the biggest performance benefit from the method.

Category	Relative Performance	Token Cost	Complexity	Best Environment
Zero-shot ICL	Baseline - Low	Low	Very low	Quick tasks, simple repositories
Few-shot / augmented ICL	Moderate	Medium	Medium	Medium-sized repositories
Prompt Engineering	Moderate-High	Low-Medium	Low	Suitable everywhere
Non-graph RACG	High	Medium	Medium	Large repositories with modular code
Graph-based RACG	Very high	High	High	Complex, interdependent repositories
Memory context management	Improves long-term performance	Low–Medium	Medium–High	Iterative development
Compression techniques	Low–Moderate	Very low	Low	Long prompts, limited tokens

Table 1 – Performance and practicality of the context engineering methods.

6.3 Industrial Adoptability of Context Engineering Methods

The reviewed context engineering methods vary significantly in their suitability for industrial deployment as indicated by the complexity column in Table 1. Techniques based on prompt engineering and zero-shot ICL are the most adoptable as they do not require additional infrastructure and can be applied and used directly in existing tools, such as GitHub Copilot [1]. Non-graph-based RACG demonstrates medium adoptability, as it relies on embedding-based retrieval with high performance boost overall.

More advanced approaches lead to lower adoptability and bigger upfront requirements. Graph-based RACG requires static analysis pipelines, ongoing maintenance of code graphs, and repository-specific tooling to create and utilize the graph representation [12]. For these reasons graph-based RACG is primarily feasible for larger organizations with large and dedicated engineering resources. Similarly, memory-based context management architectures are promising but currently lack mature, standardized implementations for practical systems. In contrast, prompt compression mechanisms and prompt engineering are highly practical and can be introduced with minimal overhead.

In summary, the most adoptable methods are those that combine prompting with retrieval and compression. While more advanced retrieval and memory systems offer superior performance, their complexity in terms of adaptability limits their immediate suitability for widespread industrial use.

7 Conclusion

This literature review has examined several context engineering methods for improving AI-assisted software development with an emphasis on the challenges posed by large industrial repositories. The examined methods were picked from the context retrieval and generation and context management foundational components of the taxonomy by Mei et al. [3]. The two foundational components analyzed mostly cover techniques that can be adopted with open source LLMs without the need to develop the LLMs directly. However, many industrial workflows have already adopted advanced tools, such as GitHub Copilot [1], into use. Accurate and definitive information on the exact implementation of such tools is not publicly available. Thus, the integration of context engineering methods and their performance boost with the agentic tools must be examined in practice. The most adoptable and useful methods in such environments that already utilize agentic LLM systems appear to be prompt engineering techniques that bring significant performance increase with very little engineering effort. Additionally, RAG systems are also very appealing. RAG systems have the possibility to substantially increase performance, especially in domain-specific tasks, with moderate cost of engineering involved with adopting the retrieval techniques.

The field of context engineering is growing very rapidly. Many papers used in this thesis are very fresh and still in the process of being peer-reviewed. Context engineering is the future for improving LLM performance with superior capabilities and performance compared to solely using prompt engineering or larger models with longer context windows in the hopes of better performance.

References

[1] "GitHub Copilot documentation," *GitHub Docs*. Available: https://docs.github.com/en/copilot. [Accessed: Sept. 27, 2025]

[2] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code." arXiv, July 14, 2021. doi: 10.48550/arXiv.2107.03374. Available: http://arxiv.org/abs/2107.03374. [Accessed: Sept. 27, 2025]

[3] L. Mei *et al.*, "A Survey of Context Engineering for Large Language Models." arXiv, July 21, 2025. doi: 10.48550/arXiv.2507.13334. Available: http://arxiv.org/abs/2507.13334. [Accessed: Oct. 22, 2025]

[4] B. Dherin, M. Munn, H. Mazzawi, M. Wunder, and J. Gonzalvo, "Learning without training: The implicit dynamics of in-context learning." arXiv, July 21, 2025. doi: 10.48550/arXiv.2507.16003. Available: http://arxiv.org/abs/2507.16003. [Accessed: Oct. 08, 2025]

[5] OpenAI, "GPT-5 System Card," Oct. 14, 2025. Available: https://openai.com/index/gpt-5-system-card/. [Accessed: Oct. 23, 2025]

[6] J. Li *et al.*, "Large Language Model-Aware In-Context Learning for Code Generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 7, Aug. 2023.

[7] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report." arXiv, Dec. 27, 2024. doi: 10.48550/arXiv.2412.19437. Available: http://arxiv.org/abs/2412.19437. [Accessed: Nov. 27, 2025]

[8] H. Jin *et al.*, "LLM Maybe LongLM: Self-Extend LLM Context Window Without Tuning," presented at the Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria, July 2024, pp. 22099–22114.

[9] Y. Tao, Y. Qin, and Y. Liu, "Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches." arXiv, Oct. 06, 2025. doi: 10.48550/arXiv.2510.04905. Available: http://arxiv.org/abs/2510.04905. [Accessed: Oct. 16, 2025]

[10]*The New Code — Sean Grove, OpenAI*, (July 11, 2025). Available: https://www.youtube.com/watch?v=8rABwKRsec4. [Accessed: Oct. 23, 2025]

[11] W. Fan *et al.*, "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*,

Barcelona, Spain, Aug. 2024, pp. 6491–6501. doi: 10.1145/3637528.3671470

[12]K. Du *et al.*, "CodeGRAG: Bridging the Gap between Natural Language and Programming Language via Graphical Retrieval Augmented Generation." arXiv, May 19, 2025. doi: 10.48550/arXiv.2405.02355. Available: http://arxiv.org/abs/2405.02355. [Accessed: Oct. 23, 2025]

[13]T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," presented at the Proceedings of the 36th International Conference on Neural Information Processing System, New Orleans, USA, Nov. 2022, pp. 16344–16359. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html. [Accessed: Oct. 22, 2025]

[14]N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," presented at the Proceedings of the 37th International Conference on Neural Information Processing Systems, New Orleans, USA, Oct. 2023, pp. 8634–8652. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html. [Accessed: Oct. 22, 2025]

[15]N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, Nov. 2023, doi: 10.1162/tacl_a_00638

[16]W. Shen *et al.*, "QwenLong-CPRS: Towards infinity-LLMs with Dynamic Context Optimization." arXiv, May 27, 2025. doi: 10.48550/arXiv.2505.18092. Available: http://arxiv.org/abs/2505.18092. [Accessed: Nov. 09, 2025]

[17]C. Packer *et al.*, "MemGPT: Towards LLMs as Operating Systems." arXiv, Feb. 12, 2024. doi: 10.48550/arXiv.2310.08560. Available: http://arxiv.org/abs/2310.08560. [Accessed: Oct. 22, 2025]

[18]R. Agrawal and H. Kumar, "Enhancing Cache-Augmented Generation (CAG) with Adaptive Contextual Compression for Scalable Knowledge Integration." arXiv, May 13, 2025. doi: 10.48550/arXiv.2505.08261. Available: http://arxiv.org/abs/2505.08261. [Accessed: Sept. 27, 2025]

[19]B. J. Chan, C.-T. Chen, J.-H. Cheng, and H.-H. Huang, "Don't Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks," in *Companion Proceedings of the ACM on Web Conference 2025*, Sydney, Australia, May 2025, pp. 893–897. doi: 10.1145/3701716.3715490

[20]J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, “A Survey on Large Language Models for Code Generation,” *ACM Trans. Softw. Eng. Methodol.*, July 2025, doi: 10.1145/3747588

[21]A. Mohsin, H. Janicke, A. Wood, I. Sarker, L. Maglaras, and N. Janjua, “Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs,” 2024, doi: 10.48550/arXiv.2406.12513. Available: https://arxiv.org/abs/2406.12513. [Accessed: Oct. 07, 2025]

[22]M. Hahn and N. Goyal, “A Theory of Emergent In-Context Learning as Implicit Structure Induction.” arXiv, Mar. 14, 2023. doi: 10.48550/arXiv.2303.07971. Available: http://arxiv.org/abs/2303.07971. [Accessed: Oct. 09, 2025]

[23]T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” presented at the Proceedings of the 34th International Conference on Neural Information Processing System, Vancouver, Canada, July 2020, pp. 1877–1901. Available: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html. [Accessed: Oct. 23, 2025]

[24]Z. Englhardt *et al.*, “Exploring and Characterizing Large Language Models for Embedded System Development and Debugging,” presented at the Conference on Human Factors in Computing Systems, Honolulu USA, May 2024, pp. 1–9. doi: 10.1145/3613905.3650764

[25]Y. Su, Y. Tai, Y. Ji, J. Li, B. Yan, and M. Zhang, “Demonstration Augmentation for Zero-shot In-context Learning,” presented at the ACL 2024, Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 14232–14244. doi: 10.18653/v1/2024.findings-acl.846

[26]D. Shrivastava, H. Larochelle, and D. Tarlow, “Repository-Level Prompt Generation for Large Language Models of Code,” presented at the Proceedings of the 40th International Conference on Machine Learning, Honolulu, USA, June 2023, pp. 31693–31715.

[27]P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, “A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications.” arXiv, Mar. 16, 2025. doi: 10.48550/arXiv.2402.07927. Available: http://arxiv.org/abs/2402.07927. [Accessed: Oct. 03, 2025]

[28]Y. Gao *et al.*, “Retrieval-Augmented Generation for Large Language Models: A Survey.” arXiv, Mar. 27, 2024. doi: 10.48550/arXiv.2312.10997. Available: http://arxiv.org/abs/2312.10997. [Accessed: Oct. 17, 2025]

[29]P. Zhao *et al.*, “Retrieval-Augmented Generation for AI-Generated Content: A Survey.” arXiv, June 21, 2024. doi: 10.48550/arXiv.2402.19473. Available: http://arxiv.org/abs/2402.19473. [Accessed: Sept. 27, 2025]

[30]F. Zhang *et al.*, “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” presented at the Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, Dec. 2023, pp. 2471–2484. doi: 10.18653/v1/2023.emnlp-main.151

[31]W. Liu *et al.*, “GraphCoder: Enhancing Repository-Level Code Completion via Coarse-to-fine Retrieval Based on Code Context Graph,” presented at the Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, USA, Oct. 2024, pp. 570–581. doi: 10.1145/3691620.3695054

[32]D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, “UniXcoder: Unified Cross-Modal Pre-training for Code Representation,” presented at the Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, Dublin, Ireland, May 2022, pp. 7212–7225. doi: 10.18653/v1/2022.acl-long.499

[33]Z. Feng *et al.*, “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” presented at the Findings of the Association for Computational Linguistics: EMNLP 2020, Online, Nov. 2020, pp. 1536–1547. doi: 10.18653/v1/2020.findings-emnlp.139

[34]Z. He, L. Karlinsky, D. Kim, J. McAuley, D. Krotov, and R. Feris, “CAMELoT: Towards Large Language Models with Training-Free Consolidated Associative Memory.” arXiv, Feb. 21, 2024. doi: 10.48550/arXiv.2402.13449. Available: http://arxiv.org/abs/2402.13449. [Accessed: Oct. 24, 2025]

[35]H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” presented at the Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, Dec. 2023, pp. 13358–13376. doi: 10.18653/v1/2023.emnlp-main.825